

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Первичный анализ и предобработка природного временного
ряда.

Студент гр. 3384

Рудаков А.Л.

Преподаватель

Мандрикова Б.С.

Санкт-Петербург

2026

Цель работы.

Повышение качества природных данных на примере наблюдений нейтронных мониторов.

Задание.

Вариант 20(3). Тикси (ТХВУ) 1-9 августа 2015

1. Скачать данные из источника за периоды и станции, указанные по вариантам в таблице. Использовать данные с одноминутным временным разрешением.
2. Выполнить обнаружение пропусков и выбросов в анализируемом временном ряде, посчитать их количество и долю от всех обрабатываемых данных.
3. Вычислить медианное значение по данным без пропусков и выбросов.
4. Заменить пропуски и выбросы медианными значениями.
5. Изобразить графики исходного и полученного временных рядов.
6. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Работа выполнялась в Jupiter Notebook, Python.

Для начала были найдены пропуски данных во временном ряду, путем сравнения полученных данных с эталонным временным рядом с шагом в 1 минуту. Суммарно найдено 59 пропусков. Доля от всех 0.005.

Далее было найдено среднее значение (109.730) и стандартное отклонение (79.651). После чего было применено правило 3-сигма (среднее $\pm$ 3 сигма), благодаря которому было найдено 198 выбросов. Доля от всех 0.015. Далее среди данных, не являющихся выбросами и пропусками была найдена медиана (97.867) и все выбросы и пропуски заполнены данным значением.

Однако на мой взгляд для данных с такими выбросами подходит не идеально, так как из-за выбросов СКО становится большим и правило отсекает

лишь самые крупные выбросы, при этом все равно остаются значения сильно выше медианных (в 3 раза). Поэтому решено добавить более жесткое правило для выбросов: IQR.

Для правила IQR был найден 25-ый перцентиль (96.4), 75-ый перцентиль (99.6), значение $IQR = 3.2$, после чего, по правилу IQR ($[Q1-IQR, Q3+IQR]$) было найдено 2102 выброса. Доля от всех 0.162. После чего найдена медиана по чистым значениям (97.733) и все выбросы и пропуски заполнены данным значением.

Графики начального временного ряда, временного ряда, после применения правила 3-сигма и временного ряда, после применения правила IQR представлены на рис.1.

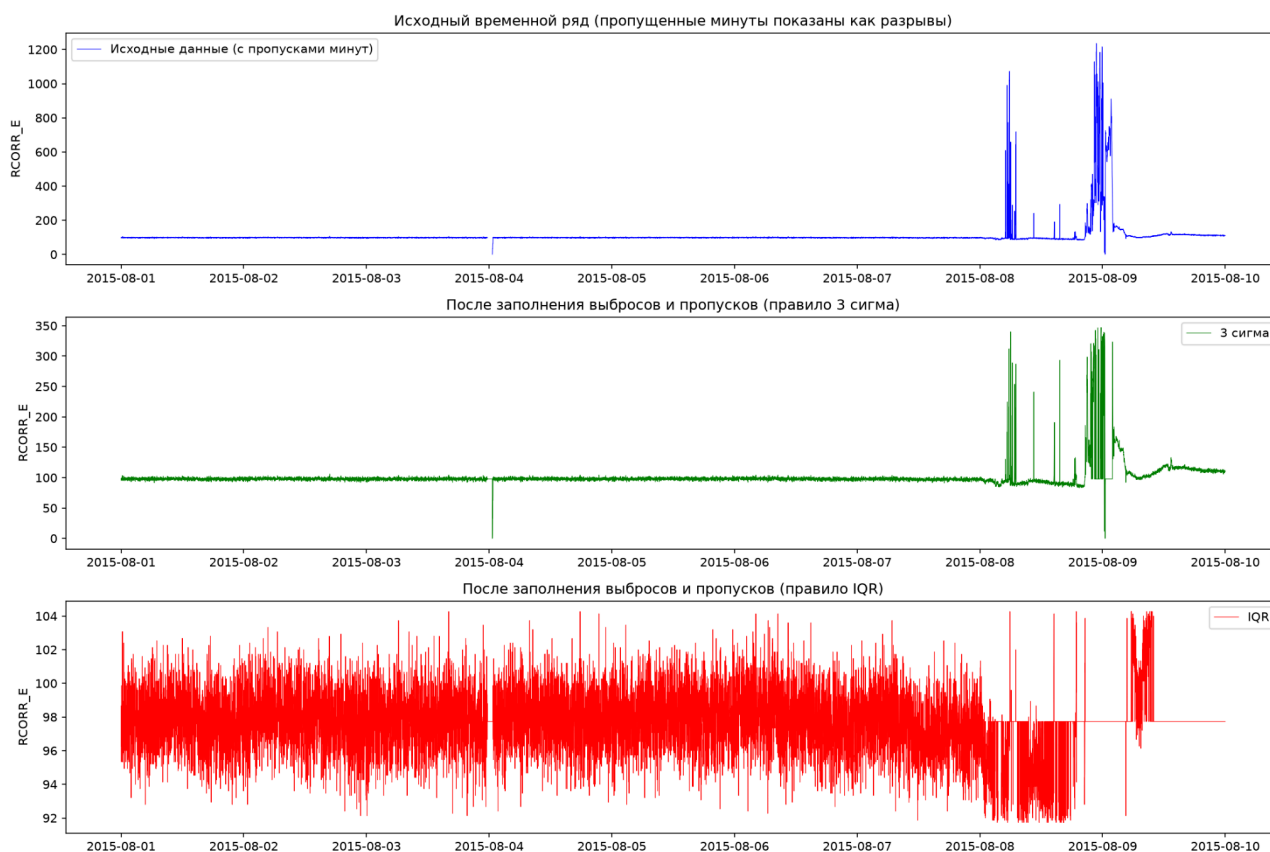


Рисунок 1 - графики полученных временных рядов

Выводы.

В ходе лабораторной работы была выполнена обработка данных нейтронного монитора станции Тикси за период с 1 по 9 августа 2015 года. Найдено 59 пропусков в данных. Данные очищены двумя методами: 3-сигма и

IQR. В первом случае доля выбросов составила 0.015, во втором случае 0.162.
Приведены графики полученных временных рядов.

ПРИЛОЖЕНИЕ А ИСХОДНЫЙ КОД

Файл `mpi_lb2.cpp`:

```
#include <iostream>
#include <mpi.h>
#include <vector>
#include <string>
#include <random>
#include <algorithm>

int main(int argc, char** argv){
    std::vector<std::vector<std::string>> values = {{ "яблоко", "каша", "помидорка"},
    {"звезда", "телефон", "бетон"} };
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib_category(0,1);
    std::uniform_int_distribution<> distrib_value(0,2);

    MPI_Init(&argc, &argv);

    int proc_rang, num_proc;

    MPI_Comm_rank(MPI_COMM_WORLD, &proc_rang);
    MPI_Comm_size(MPI_COMM_WORLD, &num_proc);

    if (num_proc > 1){
        if (proc_rang == 0){
            for (int i = 1; i < num_proc; i++){
                int category = distrib_category(gen);
                int value = distrib_value(gen);
                std::string word = values[category][value];

                MPI_Send(word.c_str(), word.length() + 1, MPI_CHAR, i, 0, MPI_COMM_WORLD);
                std::cout << "0 proccess send \"" << word << "\" to proccess " << i << "\n";

                char response[50];
                MPI_Recv(response, 50, MPI_CHAR, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                std::cout << i << " proccess send \"" << response << "\" to 0 proccess\n";
            }
        }
        else{
```

```

char response[50];
MPI_Recv(response, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

auto it = std::find(values[0].begin(), values[0].end(), response);

std::string word = (it != values[0].end()) ? "съедобное" : "несъедобное";
MPI_Send(word.c_str(), word.length() + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
}
}

MPI_Finalize();
}

```

ПРИЛОЖЕНИЕ Б
ТАБЛИЦЫ

Таблица 1 - Зависимость времени работы программы от числа процессов.

Количество процессов	Среднее время выполнения (с)
1	0.00000007
2	0.00003
3	0.00005
4	0.00008
5	0.00011
6	0.00013
7	0.00015
8	0.00017
16	0.00047
32	0.00145
64	0.00432

ПРИЛОЖЕНИЕ В

ПРИМЕРЫ РАБОТЫ ПРОГРАММЫ

Примеры работы программы:

0 process send "бетон" to process 1
1 process send "несъедобное" to 0 process
0 process send "каша" to process 2
2 process send "съедобное" to 0 process
0 process send "помидорка" to process 3
3 process send "съедобное" to 0 process
0 process send "помидорка" to process 4
4 process send "съедобное" to 0 process
0 process send "бетон" to process 5
5 process send "несъедобное" to 0 process
0 process send "гвоздь" to process 6
6 process send "несъедобное" to 0 process
0 process send "телефон" to process 7
7 process send "несъедобное" to 0 process
...
0 process send "бетон" to process 1
1 process send "несъедобное" to 0 process
0 process send "каша" to process 2
2 process send "съедобное" to 0 process
0 process send "гвоздь" to process 3
3 process send "несъедобное" to 0 process